Reading: API gateways and service discovery

Objectives

- Explain what an API gateway does for clients that use several services: one entry point, routing, and shared concerns such as authentication
- Explain service discovery and tell client-side discovery from server-side discovery
- Relate Compose's service-name DNS resolution to service discovery

Teaching

A mobile app calls three services directly, each at its own URL. The team splits `reports` into two services with new addresses.

An API gateway is one entry point in front of your services. Clients call it, and it routes each request to the right service.

It can also handle shared concerns once, such as authentication and rate limiting.

Client, gateway and two servicesA client calls one gateway, which routes to two services. client gateway orders reports

Clients know one address.

Service discovery answers one question: where is an instance right now? A service registry keeps that list.

- Client-side discovery: the caller asks the registry, then chooses an instance itself.
- Server-side discovery: the caller sends to a router at a well-known address. The router asks the registry and forwards the request.

On a Compose network, containers find each other by service name, through DNS, as in `http://api:8080`. A recreated container can get a new address, but the name stays stable. Your code queries no registry, and this is not a gateway: it adds no routing rules or authentication.

Recap

You finished 0 of 4 activities and got 0 right. Any settled activity shows what to review; Restart activity lets you try it again.

- An API gateway gives clients one entry point, routes requests and centralises shared concerns.
- Client-side discovery: the caller asks the registry. Server-side discovery: a router asks it.
- Compose service names are a simple form of discovery: a stable name, resolved by DNS to a changing address.

Cheat sheet

Reading without JavaScript: decide first, then read on.

Where to act: Choose the situation with no token first.

Next step: Ask where the shared authentication check can run so that services are never reached.

Printed alternative: Without a gateway, clients need every service address, a moved service breaks clients, and each public service repeats authentication. With a gateway, clients know one address, a moved service is a route change, and a request with no token is rejected before any service is called.

Where to act: Start with the first row.

Next step: For each row, ask who talks to the registry: the caller, or something in front of the caller.

Reading without JavaScript: decide first, then read on.

A company phone directory lists a department, not a person at a desk. When someone moves desks, callers still dial the same department name and reach the right place.

A stable name hides a changing location.

Reading without JavaScript: decide first, then read on.

- Gateway gains: decoupled clients, one place for shared concerns. Cost: one more critical hop.
- Reference services by name, not by IP address.

Sources

Status: minimum release, checked by the page author on 2026-11-08. Sources were fetched live on 2026-11-04 and the author checked the page against them; no independent education or technology

review was performed.

- API gateways: routing, aggregation, offloading
- Pattern: API Gateway
- Pattern: Server-side service discovery
- Pattern: Client-side service discovery
- Networking in Compose: service names are discoverable, addresses change

Static interaction alternatives

predict

What must change? Each option is explained after the question in this reading.

explore

Without a gateway, clients need every service address, a moved service breaks clients, and each public service repeats authentication. With a gateway, clients know one address, a moved service is a route change, and a request with no token is rejected before any service is called.

drag

Which kind of discovery is this? Each row and its explanation follow in this reading.

mcq

Compose recreates the api container, which gets a new address. reports still calls http://api:8080. Why does this work? Each option is explained after the question in this reading.